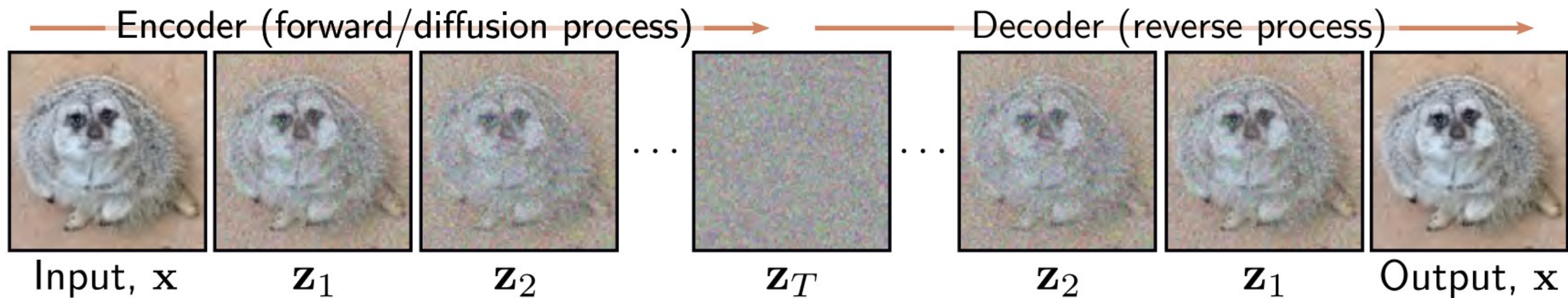


Automatic Image Analysis

Diffusion Models

Diffusion Models

- Encoder maps input to latent space by gradually mixing the data with noise until only noise remains.
- The reverse decoder process is learned and passes the data back through the latent variables removing noise at each state.



Forward Noising Process

- Given the training image set:

$$\mathcal{D} = \{x_0^{(1)}, \dots, x_0^{(N)}\}$$

- Noise schedule:

$$0 < \beta_1, \dots, \beta_T < 1$$

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

- Forward process gradually corrupts the image:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

- The tensor ε has the same shape as the image:
one Gaussian value per pixel/channel.
- It is a **concrete sampled noise tensor**.

Train NN to Predict the Noise I

- The network tries to learn the concrete noise sample from a standard Gaussian.
- However, it will not “know” the exact historical noise in an absolute sense.
- The NN

$$\varepsilon_{\theta}(x_t, t)$$

is trained to predict the best conditional estimate of the sampled noise tensor ε from the noisy image $x(t)$ at the time step t .

- The loss is usually mean squared error

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \varepsilon} \left[\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2 \right]$$

- The learned parameters belong to a time-conditioned denoising network.

Train NN to Predict the Noise II

- The training target is (c.f. slide 3)

$$\varepsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{\sqrt{1 - \bar{\alpha}_t}}$$

- So the network predicts the normalized Gaussian noise component used to corrupt x_0 into x_t .
- We know this ground truth exactly during training, because we ourselves sampled ε when constructing x_t .
- For each noisy image, the network learns:
 $(x_t, t) \mapsto \text{estimated noise inside } x_t$
- Since many clean images could correspond to a noisy image, the network learns the statistically best denoising direction under the training-image distribution.
- It does not memorize an inverse of the exact corruption process. It learns the **structure of natural images**.

Prediction of an Image (Generator)

- To generate a new image, start with pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I)$$

- Then repeatedly apply learned reverse denoising steps:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

- A typical reverse step uses the network prediction:

$$x_{t-1} \approx \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

- At each step:
 - look at the current noisy image x_t
 - ask the network to estimate the noise component
 - subtract part of that noise
 - continue until x_0 -like image space is reached.

Summary

- Training: $x_0 \longrightarrow x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$

with known sampled noise ε .

- Learn: $\varepsilon_\theta(x_t, t) \approx \varepsilon$

- Generation: $x_T \sim \mathcal{N}(0, I)$

then iteratively denoise:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

- The **training objective** is the neural network

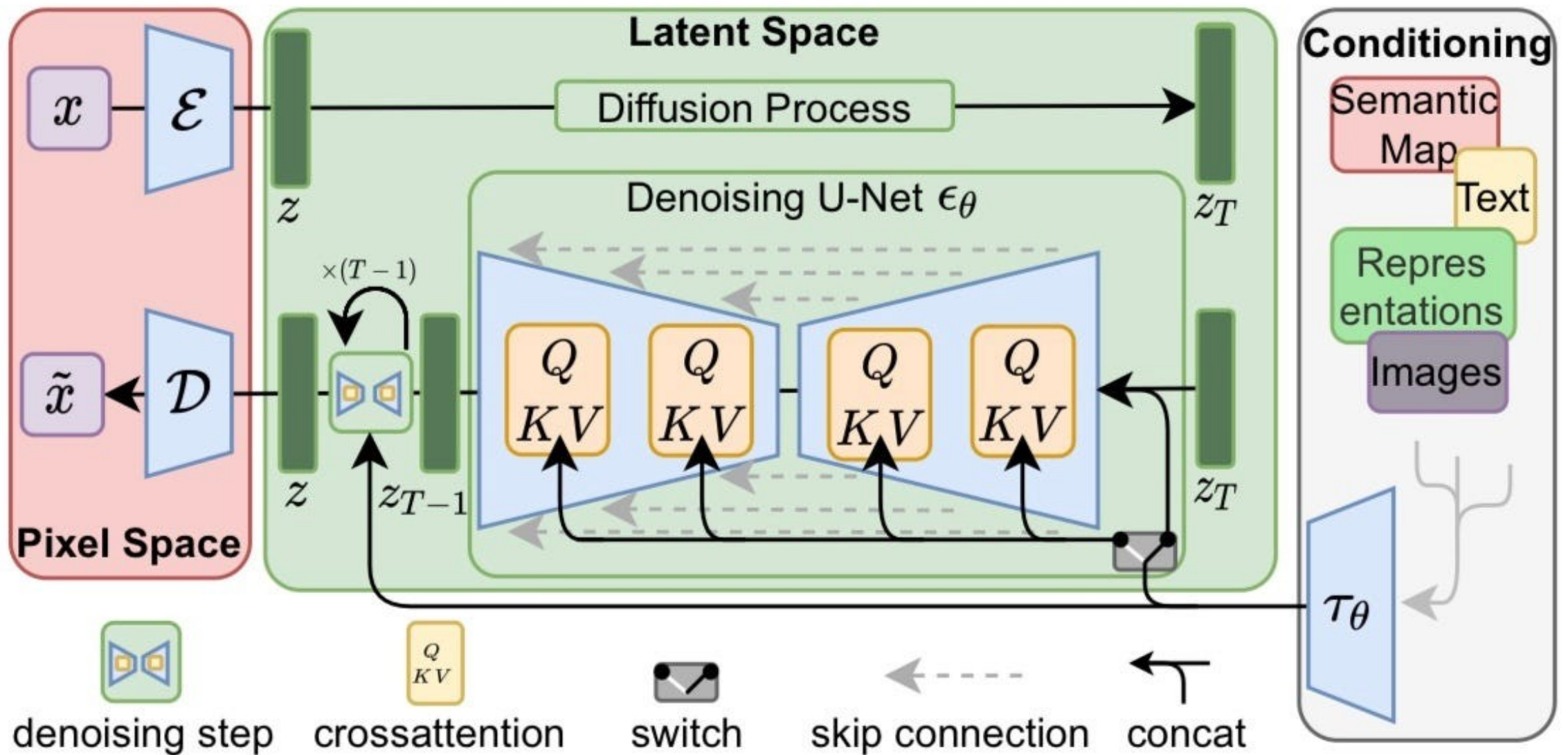
$$\varepsilon_\theta$$

that predicts, for every noise level t ,
the noise tensor that should be removed
to move a sample back toward the learned image distribution. 7

Stable Diffusion

- Stable Diffusion is generating images for text prompts.
- We want to use Stable Diffusion in a **hands-on exercise**.
- **Stable Diffusion** NN architecture:
 - Stable Diffusion contains an outer autoencoder that moves between image space and latent space,
 - and an inner U-Net-shaped denoising network that operates inside latent space.
 - The VAE performs actual compression/decompression; the U-Net performs multiscale noise prediction.
- Processing flow:
 - The **VAE** compresses the image into a latent tensor.
 - The **diffusion** noising/denoising process is **applied to that latent tensor**.
 - The **U-Net is the denoising network** operating in latent space, **conditioned by a text prompt**.
 - The **VAE decoder** is used only to convert the final denoised latent vector back into an image.

Stable Diffusion



Practical Exercise

- To enable working on simple GPU-equipped computers we focus on “Low rank adaption” (LoRA) fine-tuning:

- Suppose a pretrained Stable Diffusion layer has a weight matrix W_0 .
- Full fine-tuning would update all entries of W_0 .
- LoRA instead freezes W_0 and learns only a low-rank correction:

$$W = W_0 + \Delta W, \quad \Delta W = BA,$$

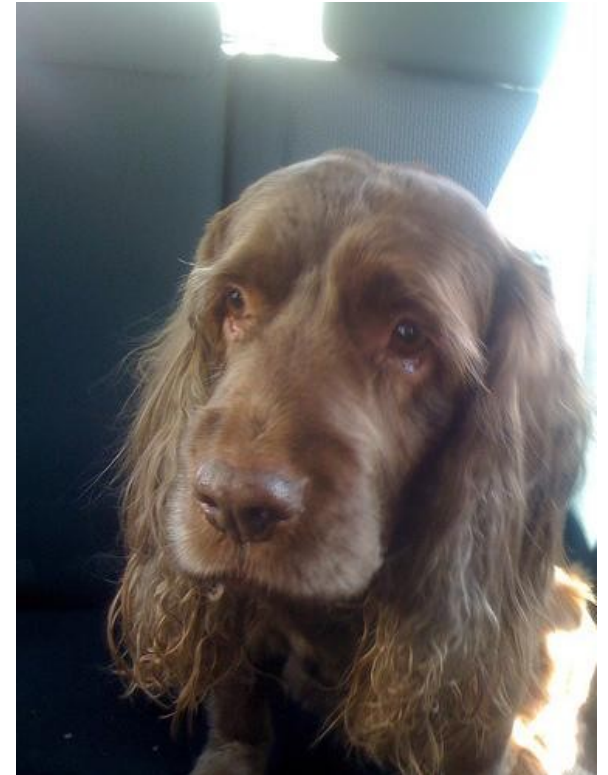
where

$$A \in \mathbb{R}^{r \times d}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(d, k)$$

- So instead of learning a full $k \times d$ update matrix, it learns two much smaller matrices.
- Reference: Smith, James Seale, et al. "Continual diffusion: Continual customization of text-to-image diffusion with c-lora." arXiv preprint arXiv:2304.06027 (2023).

Practical Exercise: Task

- Take 100 dog images as input.
- Adapt a pre-trained stable diffusion network from hugging face to predict new dog images.
- Input subset:



Practical Exercise: Processing

- ```
python finetune_lora_sd.py --pretrained_model
stable-diffusion-v1-5/stable-diffusion-v1-5
--train_dir data/dogs --output_dir dog_lora
--caption "a photo of sks dog"
--validation_prompt "a photo of sks dog running
through snow"
--max_train_steps 800 --learning_rate 1e-4
--lora_rank 8 --batch_size 1
--gradient_accumulation_steps 4 --mixed_precision
```
- ```
python predict_lora_sd.py --pretrained_model
stable-diffusion-v1-5/stable-diffusion-v1-5
--lora_dir dog_lora
--prompt "a photo of sks dog sitting in a forest,
sharp, natural light" --output_dir dog_predictions
--num_images 4 --seed 123
```

Practical Exercise: Results

- Output:

